JOHANNES KEPLER
UNIVERSITY LINZ

JOHANNES KEPLER
UNIVERSITY LINZ

Institut für Wirtschaftsinformatik
Software Engineering

Praktikum Software Engineering

Manuel Wimmer, Luca Berardinelli, Abbas Rahimi

Unit 0 – Introduction & Preliminary Discussion

JOHANNES KEPLER
UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Agenda

- Introduction
- Grouping
- Evaluation
- Tools
- Task assignment for Workshop on 17.03.2026

Goal of the Internship

- Development of an application in a team
- Specify, plan and design a software product
- Object-oriented programming and Testing (Unit tests & Code quality)
- Teamwork
- **Application of SE tools**
  - Version management (Repositories, GitHub)
  - Project management (GitHub Projects)
  - Build / Continuous Delivery (Maven + GitHub Actions)
  - Code Quality (SonarQube, SonarLint, JaCoCo)
  - Planning of the Sprints and Release Versions (GitHub Projects + Git branches)
  - Creation of System (Architecture, Code, Test cases, Documentation)

Topic: Community Analyzer

Development of a Community Analyzer Application

- A team of three developers implements this project in several sprints over 4 months
- Analyzes research communities using data from DBLP (https://dblp.uni-trier.de/xml)
- CRUD: communities, conferences, journals
- Import base data from DBLP (XML); assign conferences/journals to communities with ratings
- Query researchers active in a community based on their publications
- Import/Export: Load and save community definitions to/from file
- Analysis: Overlap of communities, track researcher publishing after leaving (BR-12)
- Filter and sort by ratings, research output (count and weighted)
- Automatic import of ratings from external sources (BR-18, BR-19)
- Graphical evaluation and reporting of analysis results

Requirements

- High-Level Requirements: see CommunityAnalyzer Requirements document (BR-01 to BR-20)
- Programming Language: **Java**
- **Technology**
  - Backend: Java
  - Frontend: JavaFX (SceneBuilder)
  - Storage: File-based or PostgreSQL (team’s choice)

Organization

- Working in teams of 3 students
- Tasks should be equally distributed considering the amount of effort
- Effort: **6 ECTS (~150 working hours)** internship and group appointments included
- LVA-leader is your Client and Advisor
- Recommendation: Completion of VL/UE Software Engineering. Java fundamentals (SW1, SW2) are assumed.
- **Each team member must participate in the implementation – equally distributed tasks!**

Time Schedule

The Software Product is being developed in **three releases**

- **Release 1:** 11 April 2026 (Saturday)
- **Release 2:** 16 May 2026 (Saturday)
- **Release 3:** 27 June 2026 (Saturday)
- **Final Submission:** 13 July 2026 (Monday)
- Submission per Release: Branch in Git with all Documentation + Code
- Each release must be tagged in Git (e.g., R1, R2, R3)

	March	**March**	March	**March**	March	April	**April**	**April**	**May**	May	**May**	**May**	May	**June**	June	June	June	**June**	**June**	**July**
	10	**17**	21	**24**	31	4	**11**	**14**	**5**	12	**16**	**19**	26	**2**	9	16	23	**27**	**30**	**13**
Day	**Tue**	**Tue**	Sat	**Tue**	Tue	Tue	**Sat**	**Tue**	**Tue**	Tue	**Sat**	**Tue**	Tue	**Tue**	Tue	Tue	Tue	**Sat**	**Tue**	**Mon**
Event	Instructions	Req. Workshop + Sprint Planning 1	**Sprint Planning 1** Completed in GitHub Projects	**Project Meeting** (individual teams)		Easter Holiday	**R1**	Release 1 Review + Sprint Planning 2	**Project Meeting** (individual teams)		**R2**	Release 2 Review + Sprint Planning 3		**Project Meeting** (Code Review)				**R3**	**Final Sprint Planning +** Final Presentation	**Final Release +** Final Documentation

Appointments – Sprint Meetings

3 Release Review + Sprint Planning Meetings

- 14.04.2026 — Release 1 Review + Sprint Planning 2
- 19.05.2026 — Release 2 Review + Sprint Planning 3
- 30.06.2026 — Final Sprint Planning + Final Presentation
- Mandatory attendance; 10 min presentation; each member must present

3 Project Meetings (individual teams)

- 24.03.2026 — Project Meeting
- 05.05.2026 — Project Meeting
- 02.06.2026 — Project Meeting (Code Review) – Feedback & Questions (30 min)

Agile Software Development

- **Iterative development (Sprints)**
  - 1 Week to max. 1 Month
  - Prioritize a set of requirements; team decides which ones to implement per sprint
  - Result of a Sprint = New version of the product
- No dedicated roles in the team
- Between 5 and 9 developers per Team
- High level of self-organization

Release 1 – 11.04.2026 | Review: 14.04.2026

Goal: UI Prototype and OO Design

Deliverables:

- First concept for building the application (Features, Components, …)
- UML Class Diagram (classes, hierarchies, patterns) with a UML Tool!
- Entity Relationship Diagram (if PostgreSQL chosen)
- UI Prototype (JavaFX / SceneBuilder)
- Continuous Integration with GitHub Actions
- Presentation of the Project Status 1 (for Sprint Planning Meeting)

Release 2 – 16.05.2026 | Review: 19.05.2026

Goal: Prototype Implementation and Unit Tests

Deliverables:

- Extended/updated UML Diagrams
- **Prototype Implementation:**
  - First version of the User Interface (JavaFX)
  - Some implemented functionality (e.g., DBLP import, CRUD communities)
- Unit Tests for individual (important) classes
- User Stories with Acceptance Criteria (linked to Software Requirements)
- Code Quality Report (max. 2 fixes proposed by SonarQube)
- Presentation of the Project Status 2 (for Sprint Planning Meeting)

Release 3 – 27.06.2026 | Review: 30.06.2026

Goal: Documentation

Deliverables:

- Extended/updated UML Diagrams; Extended Unit Tests
- **Implementation:**
  - User Interfaces; most functionalities implemented (all Features available)
- First version of the project documentation
- Final Code Quality Report (SonarQube); technical debt reduced (mandatory from R2)
- **Code coverage ≥ 85%** for all non-UI test code
- Presentation of the Project Status 3 (Sprint Planning Meeting)
- Live Demo / Screencast of the Application

Final Product – Submission: 13.07.2026

Deliverables:

- Final Project documentation
- Executable, final version of the application (on GitHub main branch, tagged)
- GitHub Documentation (README with Installation Instructions, etc.)
- Javadoc for important classes, Interfaces and Methods
- All User Stories closed with Definition of Done satisfied

Evaluation

Assessment criteria:

- Functionality of the product
- External Quality (Stability, Efficiency, User Interface)
- Internal Quality (OO Design, MVC separation, Programming Quality, API-Docs, SonarQube)
- Unit Tests (**≥85% branch coverage** for non-UI code) and Quality of Unit Tests
- Quality of the Documentation (Design, Test cases, Experience Report)
- Process Quality (GitHub Projects, Merge Requests, commit discipline, Definition of Done)
- Presentations

Tools for the Course

- Java 21, JavaFX 23.0.2
- GitHub Projects; Git (GitHub)
- Maven; GitHub Actions (CI/CD)
- UML Editor / UI Prototyping Tool
- SceneBuilder 23.0.1 (JavaFX UI design)
- Code Coverage Library: JaCoCo
- Code Quality: SonarQube (local Docker) + SonarLint (IDE plugin)
- PostgreSQL 16 + pgAdmin 4 (optional, for teams choosing database storage)

Project Organisation with GitHub Projects

- Implementation details (detailed specification) in GitHub Projects
- For each release: Software Requirements, User Stories, Tasks, Bugs
- Assign to each task a responsible person and a time estimate
- Create a Release Planning (Roadmap) in GitHub Projects
- At the end of each release, all tasks, requirements, bugs must be completed and closed
- **Merge Request per User Story is mandatory**
- Tag each release in Git (e.g., R1, R2, R3)

Source Code Management with Git

- GitHub to manage Code and Documentation
- Code must be committed in GitHub **at least 1 per Week**
- Always enter the respective id for each commit (#IssueNr) – each member must commit!
- Quality feedback via SonarQube / SonarLint – source code must be kept clean
- Document known issues that are not yet fixed
- Submission for each release must be committed in a **separate GitHub branch**

Resources

Documentation, Tutorials, Links

- TBD — GitHub Wiki / Course Repository

Data Sources

- DBLP XML: https://dblp.uni-trier.de/xml
- GII-GRIN-SCIE Conference Rating: http://gii-grin-scie-rating.scie.es/

Tool Documentation

- SonarQube: https://docs.sonarsource.com/sonarqube-community-build/
- GitHub Actions: https://docs.github.com/en/actions
- JavaFX / SceneBuilder: https://openjfx.io/

Questions: Moodle Q&A Forum (conform to https://stackoverflow.com/help/how-to-ask)

Announcements: Moodle News Forum

Next Steps

Now (10.03.2026):

- Build teams of 3 students – 1 “Team Leader” email to luca.berardinelli@jku.at
  - Subject: PR_SE2026 Team (Name, Matr.Nr, email, GitHub user)
- Distribution of topics for the Workshop

For Next week (17.03.2026):

- Get familiar with the requirements and prepare questions for the Workshop
- Plan the first version of the product and define initial responsibilities per member
- Get familiar with Git, Maven, GitHub Projects, SonarQube/SonarLint…

By 21.03.2026:

- Complete planning for Release 1 in GitHub Projects

SE Tools Workshop: 17.03.2026

- **Topic-1: Git**
- **Topic-2: Maven + GitHub Actions**
- **Topic-3: UML Tools / Editors**
- **Topic-4: UI Prototyping with SceneBuilder**
- **Topic-5: SonarQube + Code Quality**

Topic-1: Git

Git Functions and Markdown

- Create branch; Commit; Push
- Minimal example of how to resolve conflicts
- Create a Pull Request and merge

Tools: Git Bash, Git in VS Code, GitHub Desktop, SourceTree

Tutorial: https://rogerdudler.github.io/git-guide/index.de.html

Topic-2: Maven + GitHub Actions

- What is Maven? How to add dependencies to the project?
- What is GitHub Actions?
- Create a Maven Project (e.g., sum calculator)
- Create at least one Unit Test (e.g., test the sum class)
- Add JaCoCo plugin to Maven and generate code coverage report
- Compile and Test with GitHub Actions
- Execute the jar

Topic-3: UML Tools / Editors

- Explore different UML Tools
- **Suggested tools:** PlantUML, diagrams.net (draw.io), Umlet, Mermaid (GitHub-native)
- The team should explore at least 4 tools and show minimal examples
- Show functionalities (diagram creation, code generation, etc.)
- Small comparison of the tools

Topic-4: UI Prototyping with SceneBuilder

- What is JavaFX? What is SceneBuilder?
- SceneBuilder 23.0.1 is mandatory
- **Suggested tools:** Figma (free tier), Pencil Project (open-source), Penpot (open-source)
- Explore SceneBuilder and at least 3 other UI prototyping tools
- Create a minimal JavaFX UI prototype using SceneBuilder (FXML)
- Show functionalities (layout design, event binding, code generation)
- Small comparison of the tools

Topic-5: SonarQube + Code Quality

- What is SonarQube? What is SonarLint? (https://docs.sonarsource.com/sonarqube-community-build/)
- What is the difference between SonarQube and SonarLint?
- How to set up SonarQube locally (Docker)?
- Run a first analysis on a Maven project with JaCoCo code coverage
- Show an example of a code smell detected and fixed
- Integrate SonarQube analysis into GitHub Actions pipeline
- Explain Quality Gate: what it is and what happens when it fails